Theoretical Guide: RAG Systems

Foundations

Week 03 - AI Engineer Essentials Bootcamp

This guide provides an in-depth theoretical understanding of core concepts used in Retrieval-Augmented Generation (RAG) systems. It bridges the gap between academic theory and the engineering decisions made in this repository.

Table of Contents

1. TF-IDF (Term Frequency-Inverse Document Frequency)
2. Embeddings and Vector Representations
3. Similarity Metrics
4. Vector Databases
5. Retrieval Methods
6. Chunking Strategies
7. Reranking and Cross-Encoders
8. Memory in Conversational AI
9. RAG Pipeline Architecture
10. Decision Frameworks

1. TF-IDF (Term Frequency-Inverse Document Frequency)

(Implemented in `notebooks/01_naive_rag_from_scratch.ipynb`)

Background

TF-IDF is a classical information retrieval technique. It's a numerical statistic that reflects how important a word is to a document within a collection (corpus).

How It Works

TF-IDF consists of two components:

1. **Term Frequency (TF):** Measures how frequently a term appears in a document.
    - **Intuition:** If "eczema" appears 10 times in a doc, it's likely about eczema.
2. **Inverse Document Frequency (IDF):** Measures how rare a term is across all documents.

- **Intuition:** "The" appears everywhere (low importance). "Psoriasis" appears rarely (high importance).

Engineering Trade-offs

Pros	Cons
Fast: Computable on CPU in milliseconds.	**No Semantics:** "Renal" ≠ "Kidney" (synonyms fail).
Explainable: Exact keyword matches.	**Sparse:** Vectors are mostly zeros.
Zero Cost: No GPU or API keys needed.	**Out of Vocabulary:** Fails on unknown words.

---

2. Embeddings and Vector Representations

(Implemented in `notebooks/02_langchain_pdf_rag_chunking.ipynb`)

What are Embeddings?

Embeddings are dense vector representations of text where semantically similar text is mapped to close points in a high-dimensional vector space.

The "King - Man + Woman = Queen" Analogy

In a good embedding space, mathematical operations reflect semantic relationships.

> Vector("King") - Vector("Man") + Vector("Woman") ≈ Vector("Queen")

Models We Use

Engineering Note: In `src/config/config.yaml`, we default to SBERT (`all-MiniLM-L6-v2`).

- **Dimensions:** 384
- **Max Tokens:** 256 (Truncates text longer than this)
- **Size:** ~80MB (Runs fast on local CPUs)

---

3. Similarity Metrics

(Used in `notebooks/01` and `04`)

How do we calculate the "distance" between two vectors?

Cosine Similarity (Our Default)

Measures the cosine of the angle between two vectors.

- **Range:** -1 (Opposite) to 1 (Identical).
- **Why we use it:** It ignores vector magnitude (length of document) and focuses on orientation (content).

Euclidean Distance (L2)

Measures the straight-line distance between points.

- **Use case:** Clustering algorithms (K-Means).

---

4. Vector Databases

(Implemented in `artifacts/chroma/`)

Role in Architecture

A Vector DB is not just storage; it is a nearest-neighbor search engine. It indexes vectors so we don't have to calculate Cosine Similarity against every document (which is O(N) and slow).

ChromaDB (Used in this Course)

We use ChromaDB because it is:

1. **Embedded:** Runs inside your Python script (no Docker required).
2. **File-based:** Persists data to `./artifacts/chroma/` (easy to zip and share).

Engineering Note: In production with 10M+ vectors, you would migrate from embedded Chroma to Pinecone, Weaviate, or Milvus (Kubernetes). The code interface (Insert/Query) remains largely the same.

---

5. Retrieval Methods

(Implemented in `notebooks/04_advanced_retrieval_rerank_finetune.ipynb`)

Dense Retrieval (Semantic)

- **Mechanism:** Query Embedding ↔ Document Embedding.
- **Best for:** "How do I fix a rash?" (Vague, conceptual queries).

Sparse Retrieval (Keyword / BM25)

- **Mechanism:** Keyword matching (advanced TF-IDF).
- **Best for:** "Error code 0x8004" or "Patient ID 12345" (Exact matches).

Hybrid Search (The Gold Standard)

Combines Dense + Sparse scores using Reciprocal Rank Fusion (RRF).

- **Why?** It fixes the "zero result" problem of Dense search on acronyms and the "synonym problem" of Sparse search.

---

6. Chunking Strategies

(Compare these in `notebooks/02_langchain_pdf_rag_chunking.ipynb`)

Strategies Table

Strategy	Description	Best For
Fixed Size	Hard cut every N characters.	Quick prototypes; simple text.
Recursive	Splits by Paragraph → Sentence → Word.	Production (Recommended). Respects document structure.
Semantic	Uses embeddings to find "topic shifts".	Advanced research; podcasts/transcripts.

Engineering Note: We use `chunk_size: 800` and `chunk_overlap: 150` in `config.yaml`.

- **Too Small (<200):** Chunks lose context (Answer is separated from Subject).
- **Too Large (>2000):** Chunks contain multiple topics, confusing the Retriever.

---

7. Reranking and Cross-Encoders

(Implemented in `notebooks/04_advanced_retrieval_rerank_finetune.ipynb`)

Bi-Encoders vs. Cross-Encoders

1. **Bi-Encoder (Retriever)**
    - Processes Query and Document separately.

- Fast ($0.01s$).
- Lower accuracy.

2. **Cross-Encoder (Reranker)**
   - Processes Query and Document **together** (Full Self-Attention).
   - Slow ($0.5s$).
   - Very high accuracy.

The Two-Stage Architecture

We don't Rerank everything. We Retrieve 50, then Rerank the top 50 to find the best 3.

---

8. Memory in Conversational AI

(Implemented in `notebooks/03_memory_lcel_basics.ipynb`)

Why RAG Needs Memory

Retrieval finds facts, but Memory maintains flow.

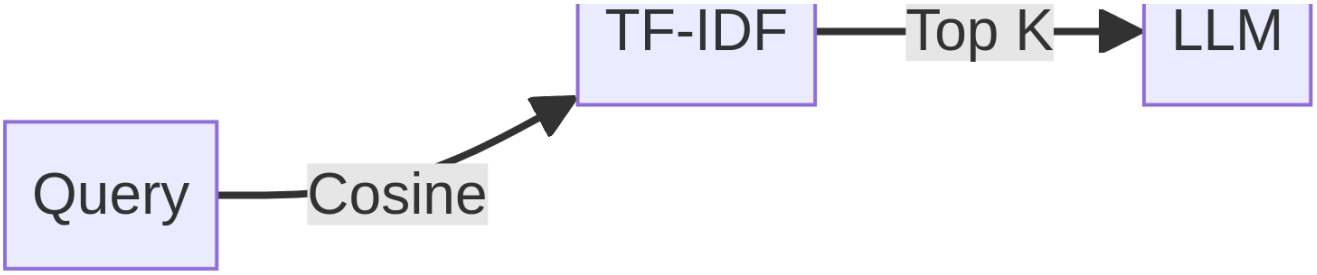
- **User:** "Who is the CEO?"
- **Bot:** "Jane Doe."
- **User:** "How old is she?"
- **Without Memory:** The bot doesn't know who "she" is.
- **With Memory:** The bot resolves "she" → "Jane Doe".

---

9. RAG Pipeline Architecture

Naive RAG (Week 3 Baseline)

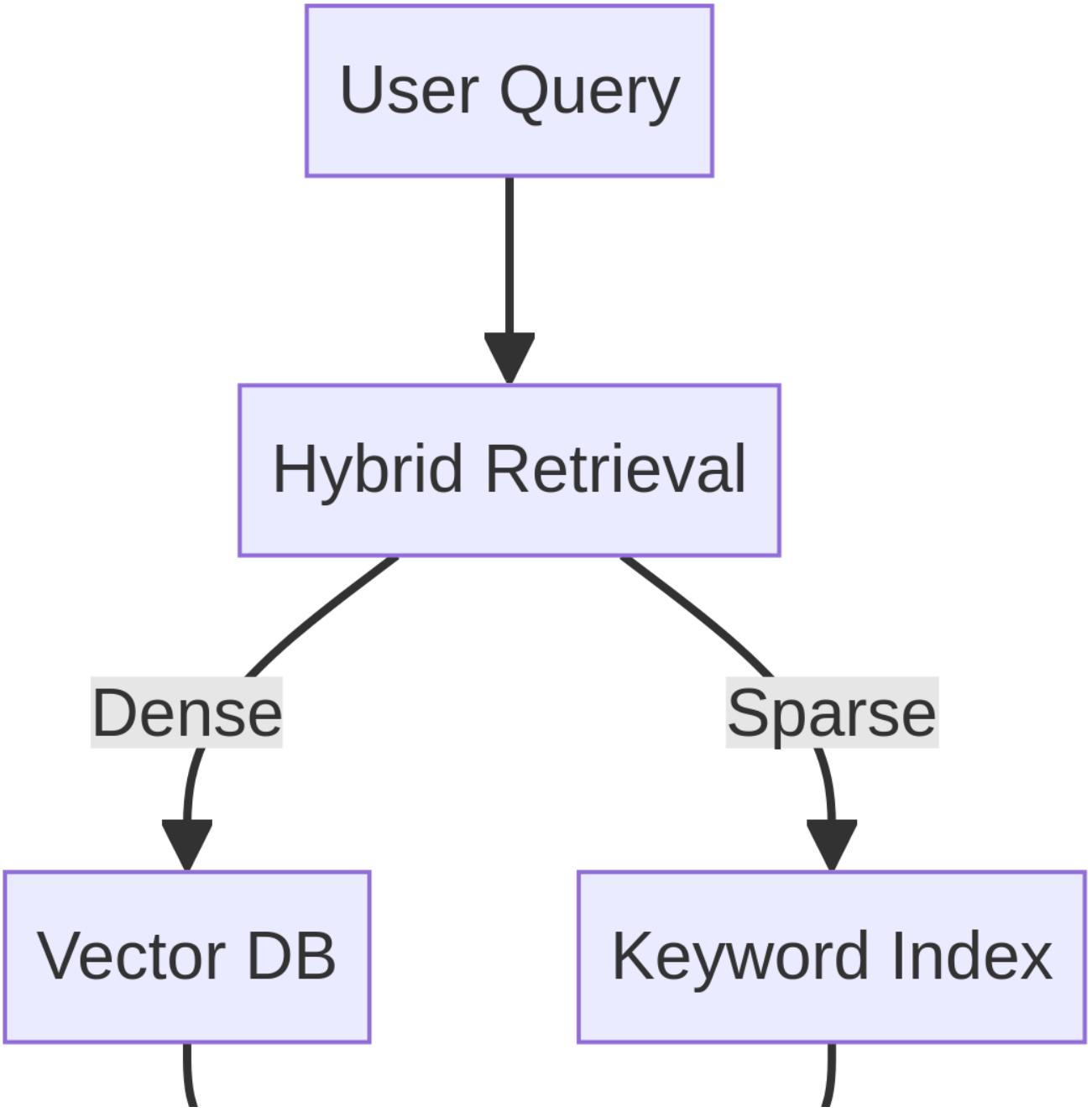
A linear path. Fast to build, brittle in production.

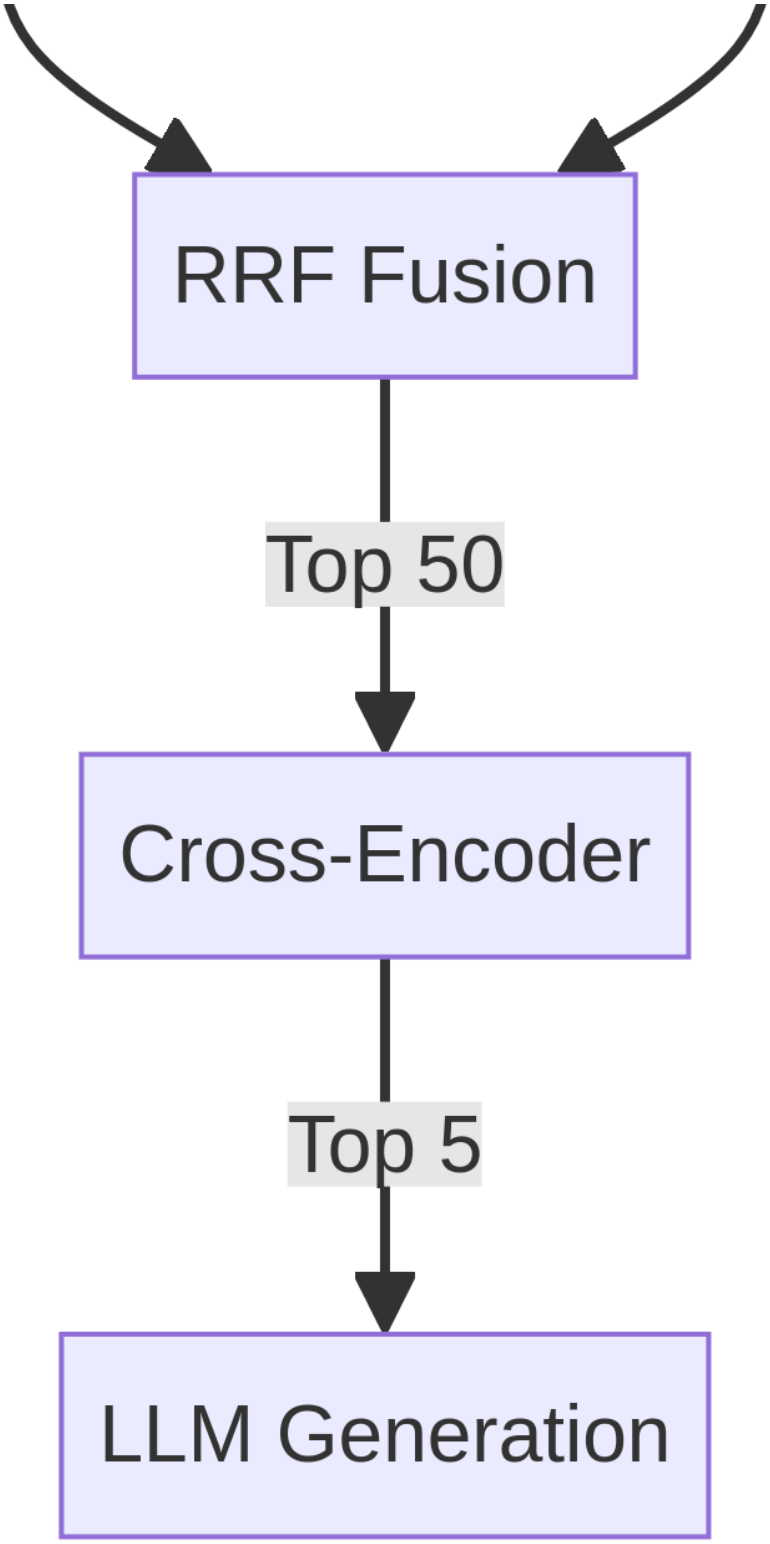


Agentic RAG (Production Standard)

A robust pipeline with feedback loops and multi-path retrieval.

Note: "Agentic" implies the system can use tools or decide "I need more info" before answering.





10. Decision Frameworks

Choosing a Retrieval Strategy

```
Plain Text
```

```
Is semantic understanding important?
├─ YES → Use dense retrieval (embeddings)
└─ NO → Use BM25
↓
Are queries ambiguous or technical?
├─ YES → Use Hybrid (Dense + Sparse)
└─ NO → Dense is sufficient
↓
Do you need highest precision?
├─ YES → Add Reranking (Cross-Encoder)
└─ NO → Stop here
```

Choosing an Embedding Model

Plain Text

```
What is your constraint?
├─ Cost/Offline → Sentence Transformers (all-MiniLM-L6-v2)
├─ Quality/Cloud → OpenAI (text-embedding-3-small)
└─ Multilingual → Cohere (embed-multilingual-v3.0)
```